```
from django.core.management.base import BaseCommand

class Command(BaseCommand):
    help = "Lists registered venues"

    def add_arguments(self, parser):
        parser.add_argument(
            "--rooms",
            "-r",
            action="store_true",
            help="Display a venue's room information",
        )

    def handle(self, *args, **options):
        for venue in Venue.objects.all():
            self.stdout.write(venue.name)

            if options["rooms"]:
                for room in venue.room_set.all():
                    self.stdout.write("    " + room.name)
```

Use the .add_arguments() method to configure command-line arguments for your management command.

The .add_argument() method on the parser registers a new command-line argument.

Using action="store_true" indicates that this flag acts as a Boolean and doesn't require any arguments to go with it.

Remember to use self.stdout.write() instead of print().

Iterate through all venues.

Iterate over the rooms associated with this venue.

If the user used --rooms, the options["rooms"] value will be True.

Exercise 2

When you delete an object with a `FileField` or `ImageField`, Django does not remove the associated file. Write a `cleanup` command that searches through all the `Musician` and `Venue` objects for their associated pictures and compares this to the uploaded files, removing any orphans. Hint 1: use a Python `set` to contain all picture references and another to contain all file references, and then use the `set.difference()` method to determine the orphan list. Hint 2: convert the picture values into `pathlib.Path` objects before comparing them. Hint 3: test your code a lot, printing out results instead of deleting files until you're absolutely sure it works.

As the `cleanup` command can be destructive (it removes files), providing a `--show` option that only shows what would be done is useful for the user. The following listing shows the setup and argument configuration for the command.

Listing E.40 Configuring arguments for the `cleanup` command

```
# RiffMates/bands/management/commands/cleanup.py
from pathlib import Path

from bands.models import Musician, Venue
from django.conf import settings
from django.core.management.base import BaseCommand, CommandError

class Command(BaseCommand):
    help = "Removes uploaded files not owned by a Musician or Venue"

    def add_arguments(self, parser):
        parser.add_argument(
```